



University
Mohammed VI
Polytechnic



Lab 3: Relational Algebra

Data Management Course

UM6P College of Computing

Professor: Karima ECHIHABI **Program:** Computer Science

Session: Fall 2026

Team Information

Member 1	Name: BENYASSINE Rayane
Member 2	Name: AIT LAHCEN Adam
Member 3	Name: BITAR Driss
Repository Link	https://github.com/Adam-Ait-Lahcen/Base-de-donn-es

1 Introduction

This report presents our solution to Lab 3. The goal is to express fifteen queries over the Research Lab Management System (RLMS) in relational algebra, using the relational schema we derived in Lab 2 from the ER model of Lab 1.

The queries cover the main families of relational algebra: selection and projection, joins, set operations (union, intersection, difference), division for "for every" questions, self-joins for "at least two" conditions, and aggregation for counting, averaging and maximum queries. For each query we gave the algebra expression (see the other file). We also summarize now the main difficulties we met and how we solved them.

2 Team Roles

The work was divided evenly. Each member wrote the expressions for five queries, and every expression was then checked by another member (schema attributes, join conditions...)

Member Group	Responsibilities
BENYASSINE Rayane	Queries Q1 to Q5; review of Q6 to Q10
AIT LAHCEN Adam	Queries Q6 to Q10; review of Q11 to Q15
BITAR Driss	Queries Q11 to Q15; review of Q1 to Q5; assembly of the report

3 Justification of the main choices

This section explains the main difficulties we met and the choices we made to solve them.

- **Explicit theta joins with qualified attribute names.** Several relations share attribute names with different meanings, for example status in both Reservation and EquipmentUnit. A natural join would silently match on them. We therefore wrote every join with an explicit condition (such as Reserves.serialNo = EquipmentUnit.serialNo) and qualified each attribute with its relation, so only the intended attributes are compared. This also handles keys whose names differ between relations (Person.pid and Reservation.madeByPid).
- **Union and intersection for "or" and "and" (Q2, Q3, Q4).** Each condition returns a set of identifiers, so "or" is a union (Q2, Q3) and "and" over two existence conditions is an intersection (Q4). In Q3 and Q4 the category is stored on the model and the laboratory on the unit, so we join EquipmentUnit with the selected models on modelId and keep labId.
- **Division for "every" (Q5, Q6).** Both queries are universal quantifications. In Q5 the dividend Supervision(pid, labId) keeps the link between a supervisor and each laboratory, and the divisor CC is the set of laboratories in the building. In Q6 the dividend is the (person, project) pairs of Participates and the divisor is the set of projects funded by a

budget that also funds laboratory 2. In both cases the dividend carries the item being checked; without it the query would only test "at least one". In Q5 a final join with Faculty returns only Faculty members and their position.

- **Counting and adding back zeros (Q7, Q13).** The grouping operator only produces groups that exist, so persons without a reservation (Q7) and laboratories without a unit (Q13) would disappear. We computed them with a set difference (No_res, No_unit) and added the constant tuple (0) to obtain a complete Total. In Q13 this makes the average run over all laboratories. The average is computed with no grouping attribute, so it is a single value, and each count is compared with it through a theta join.
- **Comparing pairs with renaming (Q7).** Two copies of Total, renamed A(pid1, cnt1) and B(pid2, cnt2), are joined with the condition cnt1 > cnt2. The comparison is strict, so equal counts and a person paired with themselves are excluded.
- **Self-join for "at least two" (Q8, Q10, Q11).** Checking "at least two different values" does not require counting. We project the (person, value) pairs into R, rename two copies (R1, R2), and join them on the person with the extra condition that the values differ (lab1 ≠ lab2 or code1 ≠ code2). Because R is a projection, duplicates are removed, so a repeated laboratory or project is counted once.
- **Approvers (Q11).** We first reduce Reservation to (approverPid, resId) after selecting the approved reservations that have an approver. The joins with Reserves and EquipmentUnit then give the (approver, laboratory) pairs, and the same self-join applies. The test on the approver uses "is not NULL", because a comparison with NULL is never true.
- **Distinct counting and maximum per group (Q14).** One reservation can include several units of the same laboratory. Projecting onto (labId, madeByPid, resId) in B removes these duplicates before counting, so each reservation is counted once per laboratory. C counts per (laboratory, person), D holds the maximum per laboratory, and the final join keeps the rows where the count equals the maximum. In a tie, all top persons are returned.
- **Negation as set difference (Q12).** A condition of the form "no reservation that..." is expressed as all persons minus those who have at least one matching reservation. This keeps persons with no reservation at all, which a selection on non-matching reservations would miss. The day condition is an interval overlap: the reservation starts before the end of 6 November and ends after its beginning.
- **Time conditions as ranges (Q9, Q15).** For September 2026 and for the year 2025 we used a half-open range on submissionTS (from the first day, up to but not including the next period), which avoids missing values late in the last day. We interpret "submitted in" and "made during" as the submission time of the reservation.
- **Threshold on counts (Q15).** We group the 2025 reservations by person, count them, and keep the persons whose count is at least